

Document 11: Comprehensive 14-Pedal Unified Architecture Specification

11.1 Architectural Framework

This document details the production-ready implementation mapping all 14 baseline processing effects directly into the single-instance, data-driven `UnifiedPedalProcessor`. By consolidating all algorithms into structural, configurable execution branches governed by runtime `.pedal` metadata, the VST achieves comprehensive tonal flexibility with absolute zero infrastructure overhead, eliminating the need for separate native plugin binaries or dynamic library files.

11.2 Core Unified Interface (C++ Header Spec)

```
#pragma once
#include <vector>
#include <string>
#include <mutex>
#include <algorithm>

enum class DspModuleType : uint8_t {
    BYPASS = 0,
    WAVESHAPER_DISTORTION,
    MODULATED_DELAY_LINE,
    BIQUAD_FILTER,
    DYNAMIC_RING_BUFFER,
    PITCH_SHIFTER_GRANULAR,
    ENVELOPE_VCA_COMPRESSOR,
    PITCH_DETECTOR_OSCILLATOR,
    DIFFUSED_DELAY_NETWORK,
    ALLPASS_FILTER_CASCADE,
    FREQUENCY_SHIFTER,
    MATHEMATICAL_WAVEFOLDER
};

struct ParameterDescriptor {
    uint16_t parameterToken;
    float minValue;
    float maxValue;
    float defaultValue;
    float currentValue;
    uint8_t targetDspNodeRegister;
};

struct PedalAssetPayload {
    uint16_t assetId;
```

```

    std::string displayName;
    std::vector<DspModuleType> activeRoutingChain;
    std::vector<ParameterDescriptor> parameters;
    std::vector<uint8_t> pixelArtBitmapBuffer;
};

class UnifiedPedalProcessor {
public:
    UnifiedPedalProcessor() : m_sampleRate(44100.0) { reset(); }
    ~UnifiedPedalProcessor() = default;

    void prepareToPlay(double sampleRate, int maxSamplesPerBlock) {
        m_sampleRate = sampleRate;
        size_t maxBufferSamples = static_cast<size_t>(sampleRate * 4.0);
        m_universalRingBuffer.resize(maxBufferSamples, 0.0f);
        m_writePointer = 0;
        reset();
    }

    void reset() {
        std::fill(m_universalRingBuffer.begin(), m_universalRingBuffer.end(), 0.0f);
        m_envelopeFollowerState = 0.0f;
        m_lfoPhase = 0.0f;
    }

    void loadPedalConfiguration(const PedalAssetPayload& payload) {
        std::lock_guard<std::mutex> lock(m_dspMutex);
        m_currentAsset = payload;
        reset();
    }

    void updateParameterFromCanvas(uint16_t token, float normalizedAccumulation) {
        for (auto& param : m_currentAsset.parameters) {
            if (param.parameterToken == token) {
                param.currentValue = param.minValue + (normalizedAccumulation * (param.maxValue -
param.minValue));
                break;
            }
        }
    }

    void processAudioBlock(float** buffer, int numChannels, int numSamples) {
        std::unique_lock<std::mutex> lock(m_dspMutex, std::try_to_lock);
        if (!lock.owns_lock()) return;

        for (int sample = 0; sample < numSamples; ++sample) {
            for (auto& node : m_currentAsset.activeRoutingChain) {
                switch (node) {

```

```

        case DspModuleType::WAVESHAPER_DISTORTION:    executeWaveshaper(buffer,
numChannels, sample); break;
        case DspModuleType::MODULATED_DELAY_LINE:    executeModulatedDelay(buffer,
numChannels, sample); break;
        case DspModuleType::BIQUAD_FILTER:          executeBiquadFilter(buffer,
numChannels, sample); break;
        case DspModuleType::DYNAMIC_RING_BUFFER:    executeDynamicRingBuffer(buffer,
numChannels, sample); break;
        case DspModuleType::PITCH_SHIFTER_GRANULAR: executeGranularPitch(buffer,
numChannels, sample); break;
        case DspModuleType::ENVELOPE_VCA_COMPRESSOR: executeVcaCompressor(buffer,
numChannels, sample); break;
        case DspModuleType::PITCH_DETECTOR_OSCILLATOR: executeSubSynth(buffer,
numChannels, sample); break;
        case DspModuleType::DIFFUSED_DELAY_NETWORK: executeDiffusedReverb(buffer,
numChannels, sample); break;
        case DspModuleType::ALLPASS_FILTER_CASCADE: executeAllpassCascade(buffer,
numChannels, sample); break;
        case DspModuleType::FREQUENCY_SHIFTER:      executeFrequencyShifter(buffer,
numChannels, sample); break;
        case DspModuleType::MATHEMATICAL_WAVEFOLDER: executeWavefolder(buffer,
numChannels, sample); break;
        case DspModuleType::BYPASS: default: break;
    }
}
}
}
}

```

private:

```

void executeWaveshaper(float** b, int c, int s);
void executeModulatedDelay(float** b, int c, int s);
void executeBiquadFilter(float** b, int c, int s);
void executeDynamicRingBuffer(float** b, int c, int s);
void executeGranularPitch(float** b, int c, int s);
void executeVcaCompressor(float** b, int c, int s);
void executeSubSynth(float** b, int c, int s);
void executeDiffusedReverb(float** b, int c, int s);
void executeAllpassCascade(float** b, int c, int s);
void executeFrequencyShifter(float** b, int c, int s);
void executeWavefolder(float** b, int c, int s);

```

```

PedalAssetPayload m_currentAsset;
std::mutex m_dspMutex;
double m_sampleRate;
std::vector<float> m_universalRingBuffer;
size_t m_writePointer;
float m_envelopeFollowerState;

```

```
float m_lfoPhase;
};
```

11.3 Comprehensive 14-Pedal Runtime Matrix

The following mapping profile maps all 14 launch pedals into configuration profiles matching the `PedalAssetPayload` framework schema:

ID	Pedal Name	Inspiration Reference	Active Routing Chain Modules	Target DSP Node Registers
0x01	Harmonic Saturation & Fuzz	ProCo RAT, EHX Big Muff, Soundtoys Decapitator	WAVESHAPER_DISTORTION	Reg 0: Drive/Saturation Level Reg 1: Symmetrical Clipping Factor
0x02	Lo-Fi Modulation & Flutter	Chase Bliss Warped Vinyl, Boss CE-2, RC-20	MODULATED_DELAY_LINE	Reg 0: Pitch Flutter LFO Depth Reg 1: Clock Rate Divider
0x03	Space Echo & Filter	Roland RE-201, EHX Deluxe Memory Man	MODULATED_DELAY_LINE + BIQUAD_FILTER	Reg 0: Echo Feedback Gain Reg 1: Resonant LP Cutoff Frequency
0x04	Half-Speed Time-Weaver	Cableguys HalfTime, Image-Line Gross Beat	DYNAMIC_RING_BUFFER	Reg 0: Slip Window Slice Size Reg 1: Linear Crossfade Intersection
0x05	Pitch-Shifting Glitch Matrix	DigiTech Whammy, Soundtoys Crystallizer	PITCH_SHIFTER_GRANULAR	Reg 0: Transposition Steps (-12/+12) Reg 1: Grain Dispersion Scatter Map
0x06	Slammed VCA Compressor	MXR Dyna Comp, Soundtoys Devil-Loc	ENVELOPE_VCA_COMPRESSOR	Reg 0: Detector Compression Threshold Reg 1: Make-up Dry/Wet Blend
0x07	Sub-Harmonic Synth & Octaver	Boss OC-2, MXR M288 Bass Octave	PITCH_DETECTOR_OSCILLATOR	Reg 0: Fundamental Pitch Tracker Cutoff Reg 1: Generated Sub Sine Mix Level
0x08	Sample-Rate Degradar	Alesis Bitrman, OTO Biscuit	WAVESHAPER_DISTORTION	Reg 0: Bit-Depth Quantization Mask Reg 1: Sample-Rate Downsampler Ratio
0x09	Granular Shimmer & Cloud	Strymon BigSky, Eventide Space	DIFFUSED_DELAY_NETWORK	Reg 0: Reverb Diffusion Decay Window

ID	Pedal Name	Inspiration Reference	Active Routing Chain Modules	Target DSP Node Registers
				Reg 1: Shimmer Pitch-Shift Factor
0x0A	Formant Vocal Shifter & Wah	Mu-Tron III, Soundtoys Little AlterBoy	ENVELOPE_VCA_COMPRESSOR + BIQUAD_FILTER	Reg 0: Dynamic Envelope Mapping Node Reg 1: Formant Filter Formant Center
0x0B	Tape-Stop & Reverse Echo	Danelectro Back Talk, Illformed GlitchTape	DYNAMIC_RING_BUFFER	Reg 0: Decay Braking Speed Curve Reg 1: Memory Playback Inversion Flag
0x0C	Resonant Multi-Phase Matrix	MXR Phase 90, Mu-Tron Bi-Phase	ALLPASS_FILTER_CASCADE	Reg 0: Cascade Sweeping Allpass Center Reg 1: Modulating LFO Frequency Speed
0x0D	Frequency Shifter & Ring Mod	Moog MF-102, Bode Frequency Shifter	FREQUENCY_SHIFTER	Reg 0: Single-Sideband Multiplier (Hz) Reg 1: Direct Ring Modulation Carrier
0x0E	Asymmetrical Wavefolder	Intellijel Bifold, Modular Eurorack Waveshapers	MATHEMATICAL_WAVEFOLDER	Reg 0: Input Amplification Fold Boundary Reg 1: Recurse Wave-Folding Iteration Limit